

Flash Sale — Gestion de Stock

Réservation courte + Anti-survente (Redis NoSQL)

Module : Bases de Données NoSQL
SGBD imposé : Redis (Base clé-valeur)

Étudiant (Chef de projet) : Khalid Morjan
Co-équipier : Anas Lahraoui
Filière : Systèmes d'Information & Intelligence Artificielle
Année universitaire : 2025–2026

Livrables : service (`app.py`) + couche Redis (`redis_client.py`) + config (`config.py`)
scripts de simulation (`simulate_clients.py`, `scenario_concurrency.py`) + logs
(`results_logs.txt`)
installation (`requirements.txt`) + Gantt + annexes.

Date : 10 février 2026

Table des matières

1	Résumé exécutif	3
2	Contexte, objectifs et périmètre	3
2.1	Contexte	3
2.2	Objectifs	3
2.3	Contraintes	4
2.4	Périmètre du projet	4
3	Méthodologie Agile / Scrum	5
3.1	Organisation Scrum (4 semaines)	5
3.2	Rôles	5
3.3	Backlog (extraits)	5
3.4	Plan des sprints (résumé)	10
4	Architecture de la solution	12
4.1	Vue en couches	12
4.2	Schéma global	13
4.3	Choix techniques Redis	13
5	Modèle de données Redis (clés & structures)	14
5.1	Principe de gestion du stock réservé	14
5.2	Schéma des clés et relations	15
5.3	Liste des clés	15
5.4	Justification (structures)	16
6	Implémentation des cas d'usage	16
6.1	Initialisation du stock	16
6.2	Réservation atomique (Lua)	17
6.3	Annulation (cancel)	18
6.4	Expiration et libération automatique (sweeper)	18
6.5	Paieement confirmé (idempotence)	19
7	Gestion des cas limites	20
7.1	Survente (Race Condition)	20
7.2	Double paiement	20
7.3	Requêtes répétées / retries	20
7.4	Abandon tardif du client	21
7.5	Données manquantes ou incohérentes	21
7.6	Résumé	21
8	Installation et exécution (Windows + Docker)	22
8.1	Prérequis	22
8.2	Démarrage du serveur Redis (Docker)	22
8.3	Configuration de l'authentification Redis (ACL)	22
8.4	Variables d'environnement (PowerShell)	23
8.5	Vérification de l'application (Health Check)	23
8.6	Commandes principales de démonstration	23

9 Tests, simulation de charge et preuves	24
9.1 Simulation multi-clients (<code>simulate_clients.py</code>)	24
9.2 Comparatif SAFE vs UNSAFE (<code>scenario_concurrency.py</code>)	24
9.3 Interprétation des résultats	25
9.4 Logs et preuves	25
10 Planification du projet (Diagramme de Gantt)	25
11 Conclusion	26
A Annexes	26
A.1 Arborescence du projet	26
A.2 Commandes de démo	27

1 Résumé exécutif

Ce projet répond au besoin d'une marque effectuant des ventes flash (durée minutes/heures) où le risque majeur est la **survente** due aux accès concurrents (achats simultanés) et aux paniers non finalisés. La solution proposée repose sur **Redis** (base clé-valeur NoSQL) et met en œuvre :

- une **réservation temporaire** avec **TTL** (expiration automatique) ;
- une **atomicité** (script Lua) pour empêcher les conditions de course ;
- des mécanismes de **libération du stock** après expiration (sweeper) ;
- des **métriques** et un **audit** pour prouver la cohérence, mesurer l'abandon et tracer les opérations ;
- l'**idempotence** contre le double paiement.

2 Contexte, objectifs et périmètre

2.1 Contexte

Les **ventes flash** sont une stratégie très utilisée dans le e-commerce : une quantité limitée de produits est proposée pendant une durée très courte (quelques minutes à quelques heures). Ce modèle provoque :

- un **pic de trafic** important ;
- un grand nombre d'actions **simultanées** (concurrence) ;
- une forte pression sur la **gestion du stock** et la cohérence des données.

Dans ce contexte, le risque technique majeur est la **survente** (*overselling*). Elle apparaît lorsque plusieurs clients lisent le stock disponible au même moment, pensent tous que le produit est encore disponible, puis effectuent des opérations concurrentes. Sans mécanisme robuste, la quantité **réservée** peut dépasser la quantité **réelle**, entraînant un stock négatif et des annulations.

Un second problème critique est celui des **paniers non finalisés** : de nombreux clients réservent un produit, mais n'effectuent pas le paiement. Sans expiration, le stock reste bloqué artificiellement, ce qui diminue les ventes réelles et pénalise les clients sérieux.

Ainsi, un système de vente flash doit gérer simultanément :

- la **concurrence** (race conditions, délais, accès simultanés) ;
- la **réservation temporaire** et l'**expiration automatique** ;
- l'**idempotence** pour éviter les doubles paiements ;
- la **traçabilité** (audit) et les **indicateurs** pour mesurer la qualité du système.

2.2 Objectifs

Le projet poursuit des objectifs **fonctionnels**, **techniques** et **analytiques**.

Objectif principal Garantir la **cohérence du stock** pendant une vente flash, même sous forte charge et accès concurrents.

Objectifs fonctionnels

- **Empêcher la survente** sous forte concurrence.
- **Réserver temporairement** du stock au passage en caisse (panier).
- **Libérer automatiquement** le stock si le client abandonne (expiration).
- **Confirmer le paiement** pour transformer une réservation en vente définitive.
- **Permettre l'annulation** afin de relâcher immédiatement une réservation.

Objectifs techniques

- Utiliser Redis comme base NoSQL **clé-valeur** pour assurer performance et simplicité.
- Garantir l'**atomicité** des opérations critiques (vérification + mise à jour) afin d'éliminer les conditions de course.
- Exploiter les **TTL** (*Time To Live*) pour gérer l'expiration des réservations.
- Assurer l'**idempotence** lors de la confirmation du paiement (protection double paiement / retries).

Objectifs analytiques (KPI) Le système doit produire des indicateurs mesurables pour valider son comportement :

- **Taux d'abandon** (réservations expirées / réservations créées) ;
- **Stock réservé vs stock vendu** ;
- **Incidents** détectés (incohérences, cas limites) ;
- **Audit** des opérations pour la traçabilité (preuves dans le rapport).

2.3 Contraintes

- **Redis obligatoire** (structures Redis + TTL).
- Prévention des incohérences : **atomicité** des opérations et gestion des cas limites (double paiement, retries, abandon tardif).
- **Simulation de charge** multi-clients concurrents pour démontrer l'anti-survente.
- Travail en mode **Agile/Scrum** : backlog, sprints, rôles, livrables, preuves (Jira).

2.4 Périmètre du projet

Inclus

- Gestion du stock par produit, réservation temporaire, confirmation, annulation et expiration.
- Anti-survente par opérations atomiques (scripts Lua).
- Indicateurs (KPI) et journal d'audit.
- Scripts de simulation et scénarios SAFE vs UNSAFE (preuve expérimentale).

Exclus Pour rester concentré sur le coeur NoSQL et la cohérence concurrente, les éléments suivants ne sont pas traités :

- interface Web / API REST complète ;
- paiement réel (intégration banque/PSP) ;
- haute disponibilité Redis Cluster, multi-entrepôts.

3 Méthodologie Agile / Scrum

3.1 Organisation Scrum (4 semaines)

Afin de livrer une solution **démontrable, testable et justifiable** (preuve anti-survente + indicateurs), le projet a été piloté en **Agile/Scrum** sur **4 sprints** (1 semaine par sprint). Cette organisation a permis :

- une **priorisation** claire des fonctionnalités critiques (anti-survente, TTL, idempotence) ;
- des **livraisons incrémentales** à chaque fin de sprint (fonctionnalités + preuves) ;
- une **traçabilité** via Jira (Epics, User Stories, tâches, statuts) ;
- une validation continue grâce à des **tests et simulations** multi-clients.

Cadence Scrum Chaque sprint suit un cycle court et rigoureux :

- **Sprint Planning** : sélection des User Stories prioritaires et définition des critères d'acceptation ;
- **Daily** (court) : suivi d'avancement, blocages, réallocation des tâches ;
- **Sprint Review** : démonstration des fonctionnalités livrées (CLI + logs + KPI) ;
- **Rétrospective** : amélioration (robustesse, lisibilité, stabilité, documentation).

3.2 Rôles

- **Product Owner / Chef de projet : Khalid Morjan**. Responsabilités : cadrage du besoin métier, définition des Epics, priorisation du backlog, rédaction du rapport, gestion Jira (sprints, tickets), préparation de la soutenance et validation des critères d'acceptation.
- **Développeur : Anas Lahraoui**. Responsabilités : implémentations techniques (scripts Redis/Lua, fonctions de service), tests fonctionnels, stabilisation, contribution à la documentation (installation, démonstration).

3.3 Backlog (extraits)

Le backlog produit est structuré autour de **5 Epics** correspondant aux exigences principales du projet.

Epic	User Story (formulation)	Priorité	Statut
E1	En tant que système , je veux un modèle de clés Redis (stock, réservations, index) afin d'assurer une base fiable pour éviter la survente.	Haute	Done
E2	En tant que client , je veux réserver temporairement un stock avec un TTL afin de sécuriser mon panier sans bloquer le stock indéfiniment.	Haute	Done
E3	En tant que client , je veux confirmer le paiement de manière idempotente afin d'éviter le double paiement et les incohérences en cas de retry.	Haute	Done

E4	En tant que chef de projet , je veux simuler une charge concurrente et comparer SAFE vs UNSAFE afin de prouver expérimentalement l'anti-survente.	Haute	Done
E5	En tant qu' auditeur , je veux des métriques (KPI) et des logs exploitables afin de mesurer l'abandon, tracer les opérations et valider la cohérence.	Moyenne	Done

Work				
Sprints				
☐	▼	<u>FSR-1 Epic 1 – Conception & règles métier</u>		DONE
☐		 FSR-11 Justification du choix Redis		DONE 
☐		 FSR-18 Analyse du problème métier (flash sale & s...	60	DONE 
☐		 FSR-14 Conception du workflow global	60	DONE 
☐		 FSR-48 Définition du modèle de données Redis pour...		DONE 
☐		 FSR-15 Identification des risques et cas limites		DONE 
☐		 FSR-12 Définition des règles métier		DONE 
☐	▼	<u>FSR-23 Epic 2 – Réservation et Expiration du Stock</u>		DONE
☐		☒ FSR-29 Documentation du Sprint 2 et validation d...	60	DONE 
☐		☒ FSR-24 Conception du modèle de clés Redis pour...	60	DONE 
☐		☒ FSR-25 Définition des règles d'expiration des rése...	60	DONE 
☐		☒ FSR-28 Simulation de plusieurs clients réservant s...	60	DONE 
☐		☒ FSR-27 Gestion de l'expiration et libération autom...	60	DONE 
☐		☒ FSR-26 Implémentation de la réservation tempora...	60	DONE 
☐	▼	<u>FSR-49 Epic 3 – Confirmation paiement & Indicateurs</u>		DONE
☐		 FSR-58 Confirmation paiement idempotente		DONE 
☐		 FSR-51 Indicateurs de performance (KPI)		DONE 
☐		 FSR-52 Audit des opérations		DONE 
☐		 FSR-53 Test du double paiement		DONE 
☐	▼	<u>FSR-54 Epic 4 – Simulation de charge & preuve anti-survente</u>		DONE
☐		 FSR-63 Simulation multi-clients		DONE 
☐		 FSR-64 Comparaison SAFE vs UNSAFE		DONE 
☐		 FSR-65 Analyse des résultats		DONE 

Preuve Jira (Backlog)

3.4 Plan des sprints (résumé)

Les sprints ont été conçus pour livrer d'abord le **coeur critique** (anti-survente) puis la **preuve** (simulation + KPI).

— **Sprint 1 (Semaine 1) – Cadrage + socle Redis + organisation Jira**

Livrables : modèle de données (clés/structures), initialisation stock, configuration Docker/ACL, premier service CLI (ping/init/status), backlog Jira structuré.

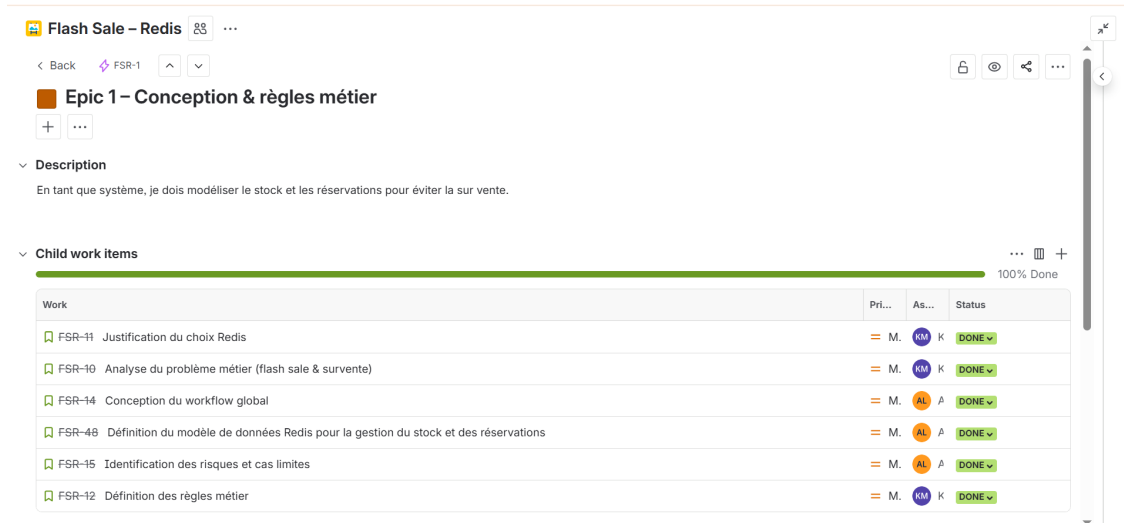


FIGURE 2 – Capture Jira – Sprint 1 : cadrage métier, modèle Redis, setup et premières commandes CLI

— **Sprint 2 (Semaine 2) – Réservation TTL + anti-survente (atomicité Lua)**

Livrables : réservation atomique (Lua), TTL, index actifs, annulation, libération automatique (sweeper), premiers tests de cohérence.

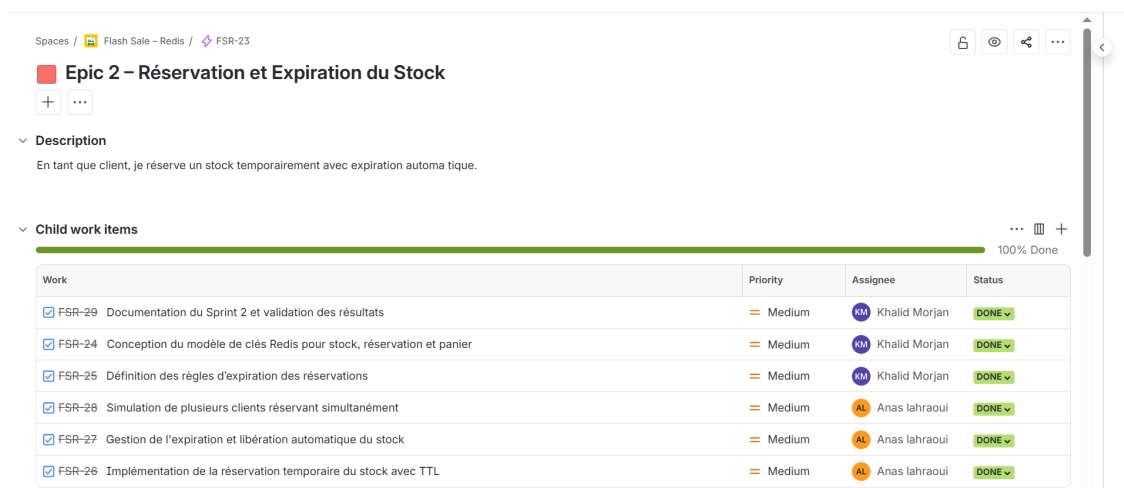


FIGURE 3 – Capture Jira – Sprint 2 : réservation TTL, atomicité Lua, annulation et libération du stock

- **Sprint 3 (Semaine 3) – Confirmation paiement + idempotence + KPI**
Livrables : confirmation idempotente (clé de verrou SET NX EX), mise à jour des compteurs (sold/abandon/incidents), audit enrichi, scénarios de test.

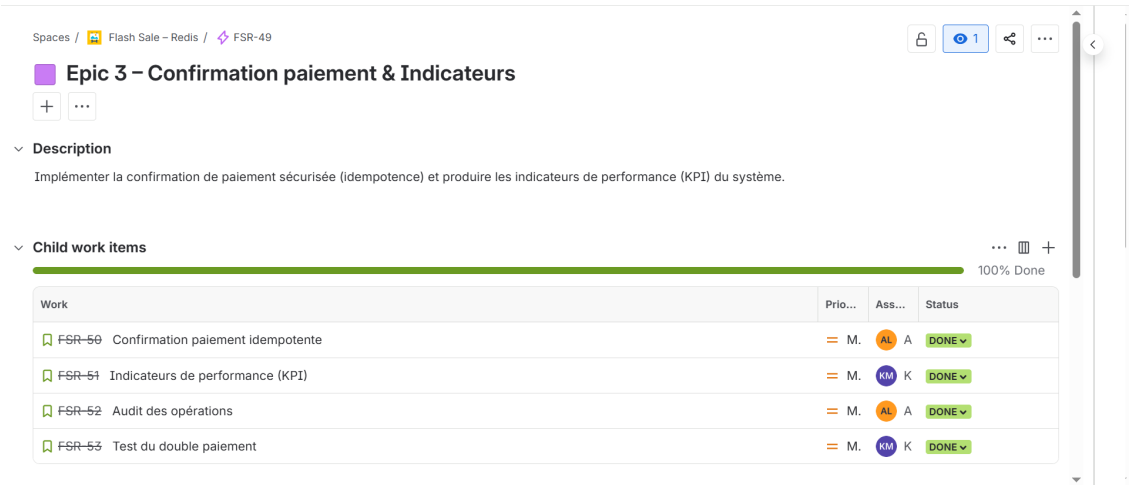


FIGURE 4 – Capture Jira – Sprint 3 : confirmation de paiement idempotente, métriques et audit

- **Sprint 4 (Semaine 4) – Simulation, preuve SAFE vs UNSAFE, stabilisation et livraison**
Livrables : scripts de simulation multi-clients, scénarios concurrents SAFE/UNSAFE, collecte des résultats, stabilisation, documentation d’installation, préparation soutenance.

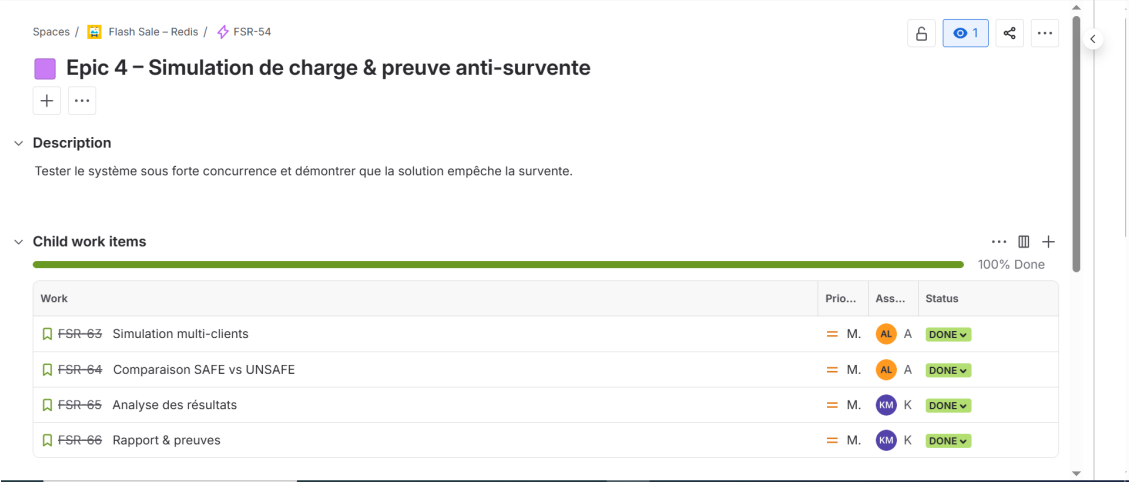


FIGURE 5 – Capture Jira – Sprint 4 : simulations, preuve anti-survente, stabilisation et livrables

— Sprint 5 (Qualité & Observabilité) – Logs, preuves et annexes

Objectif : mettre en place tous les mécanismes permettant d'**observer, mesurer et prouver** le bon fonctionnement du système de vente flash.

Livrables : extraction de logs (MONITOR), résultats structurés, tableaux KPI, annexes (captures Jira, commandes de démo, preuves), consolidation du rapport.



FIGURE 6 – Capture Jira – Sprint 5 : observabilité (logs), KPI, preuves et annexes du rapport

4 Architecture de la solution

4.1 Vue en couches

L'architecture du projet a été conçue selon un modèle en **couches logiques** afin d'assurer :

- une **séparation claire des responsabilités**,
- une meilleure **maintenabilité**,
- une **lisibilité du code**,
- une facilité de **test et d'évolution**.

Le système est structuré en trois couches principales :

— config.py – Couche configuration

Cette couche centralise tous les paramètres techniques :

- adresse Redis (host, port, base),
- authentification ACL (username/password),
- durées de TTL (réservation, verrous de paiement),
- intervalles du sweeper.

Elle permet d'adapter le système à différents environnements sans modifier le code métier.

— redis_client.py – Couche d'accès aux données

Cette couche joue le rôle de **repository** :

- encapsulation des clés Redis,

- exécution des scripts Lua,
- opérations atomiques (check+update),
- manipulation des structures Redis (String, Set, Hash, List).

Elle garantit que toutes les interactions avec Redis respectent les règles de cohérence.

— `app.py` – Couche service / orchestration

Cette couche représente le **service applicatif** accessible via CLI :

- initialisation du stock,
- réservation,
- confirmation de paiement,
- annulation,
- sweeper (libération),
- métriques et audit.

Elle orchestre la logique métier en utilisant la couche Redis.

Cette architecture sépare clairement :

Configuration → *AccsDonnes* → *LogiqueMtier*

4.2 Schéma global

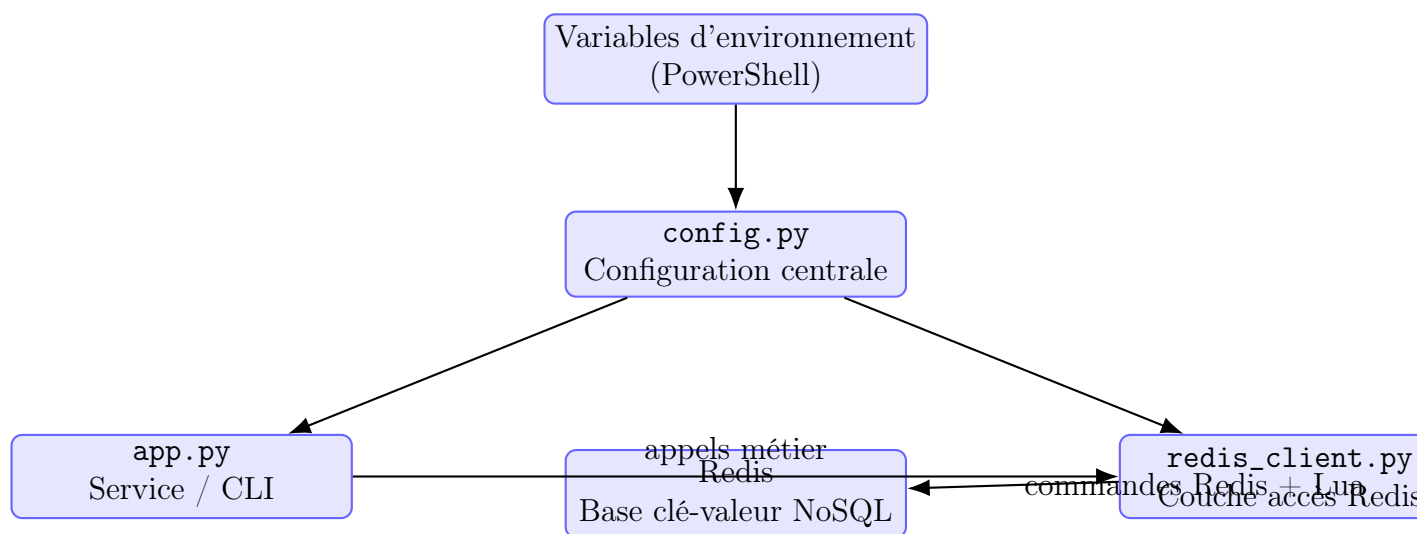


FIGURE 7 – Architecture logique du système Flash Sale basé sur Redis

4.3 Choix techniques Redis

Le choix de Redis est motivé par sa rapidité et ses fonctionnalités adaptées aux systèmes concurrents.

— **TTL (Time To Live)**

Permet l'expiration automatique des réservations afin d'éviter le blocage du stock en cas d'abandon panier.

— Scripts Lua

Les scripts Lua sont exécutés directement dans Redis, garantissant :

- l'**atomicité** (aucune autre opération ne s'intercale),
- la suppression des **race conditions**,
- la cohérence du stock sous forte concurrence.

— Structures Redis utilisées

- **String** : compteurs (stock, metrics),
- **Set** : index des réservations actives,
- **Hash** : stockage des quantités associées,
- **List** : journal d'audit chronologique.

Cette combinaison permet d'obtenir un système :

rapide + cohérent + résistant à la concurrence + mesurable

5 Modèle de données Redis (clés & structures)

5.1 Principe de gestion du stock réservé

Le modèle de données a été conçu pour répondre à une contrainte centrale des ventes flash : **empêcher la survente** tout en gérant les **paniers non finalisés**. Pour cela, nous distinguons deux notions :

- **Stock total** : quantité disponible au lancement de la vente flash.
- **Stock réservé** : quantité temporairement bloquée par des paniers en cours (réservations).

Le principe retenu (robuste et adapté à Redis) est de calculer la disponibilité de manière déterministe :

$$available = stock_total - stock_reserved$$

où **stock_reserved** représente les unités bloquées tant que la réservation est active (non expirée et non confirmée).

Pourquoi ce modèle ?

- Il permet une gestion simple via des **compteurs Redis** (INCRBY/DECRBY).
- Il évite des opérations coûteuses et garantit une lecture rapide.
- Il s'adapte parfaitement au mécanisme de **TTL** (expiration automatique).

Cycle de vie du stock

- **Reserve** : on vérifie **available** et on incrémente **stock_reserved**.
- **Confirm** : on décrémente **stock_reserved** et on incrémente **metrics:sold**.
- **Cancel / Expire** : on décrémente **stock_reserved** et on incrémente **metrics:abandon**.

5.2 Schéma des clés et relations

Le schéma suivant illustre les clés Redis principales et leurs interactions (stock, réservations, index actifs, audit et métriques). L'ensemble est optimisé pour :

- **performance** (accès $O(1)$ majoritairement),
- **cohérence** sous concurrence,
- **traçabilité** (audit),
- **observabilité** (KPI).

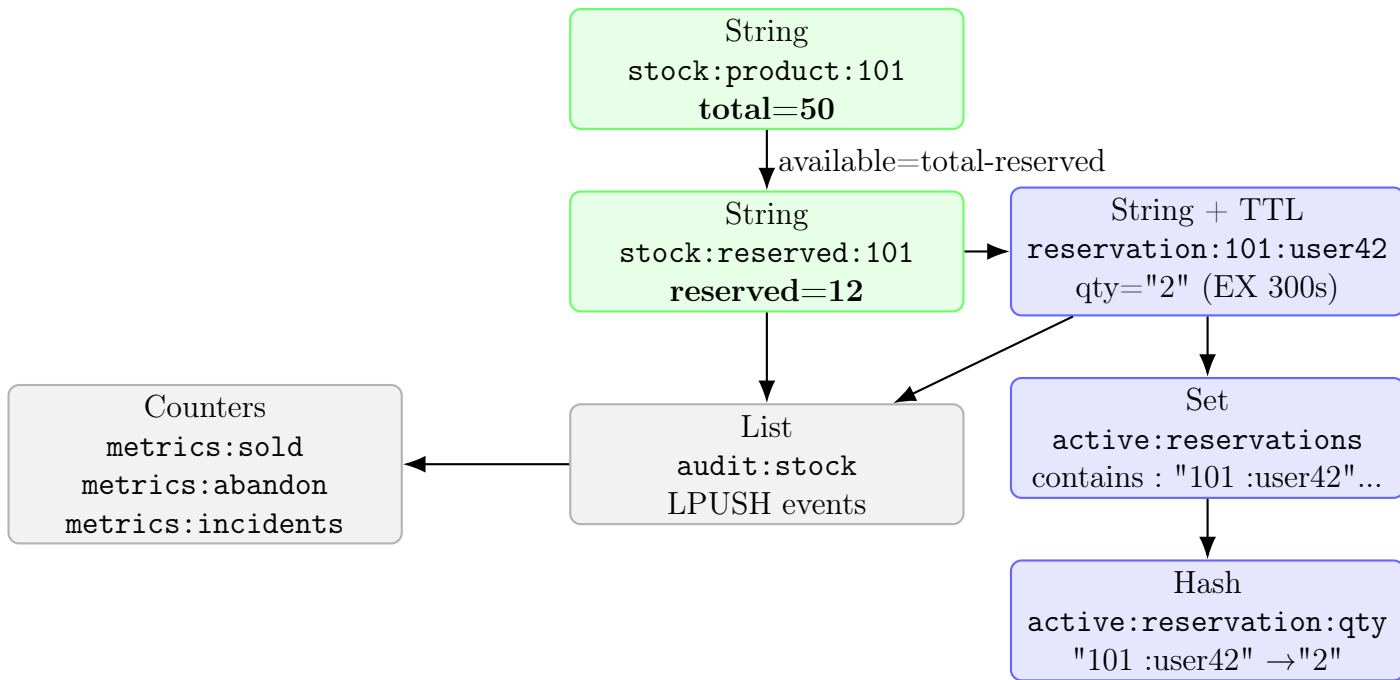


FIGURE 8 – Modèle de clés Redis : stock, réservation TTL, indexation, métriques et audit

5.3 Liste des clés

Le tableau ci-dessous présente les principales clés utilisées, leurs types Redis et leur rôle dans le système.

Clé	Type	Rôle
<code>stock:product:<id></code>	String	Stock total (capacité initiale) du produit
<code>stock:reserved:<id></code>	String	Compteur du stock réservé (bloqué par paniers)
<code>reservation:<id>:<user></code>	String + TTL	Quantité réservée par utilisateur, expire automatiquement
<code>active:reservations</code>	Set	Index des réservations actives (<code>id:user</code>) pour sweeper
<code>active:reservation:qty</code>	Hash	Quantité associée à chaque réservation <code>id:user</code>

<code>payment:processed:<order></code>	String + TTL	Clé idempotente empêchant le double paiement
<code>metrics:sold</code>	String counter	Nombre de ventes confirmées (paiements)
<code>metrics:abandon</code>	String counter	Nombre d'abandons (expiration TTL ou libération)
<code>metrics:incidents</code>	String counter	Anomalies détectées (ex. données manquantes)
<code>audit:stock</code>	List	Journal chronologique des opérations pour preuves

5.4 Justification (structures)

Le choix des structures Redis répond à des objectifs de performance et de cohérence :

- **String** : idéal pour les **compteurs** (INCR/DECR) et valeurs simples (stock, métriques).
- **Set** : fournit une liste **unique** des réservations actives, pratique pour le sweeper (scan sans doublons).
- **Hash** : permet d'associer rapidement une réservation à sa quantité, utile lors des libérations automatiques.
- **List** : stocke un **audit chronologique** exploitable dans le rapport (preuves de cohérence et traçabilité).

Résultat attendu Grâce à ce modèle, le système garantit :

- **aucune survente** (stock jamais négatif),
- une **libération automatique** en cas d'abandon,
- une **observabilité complète** via KPI et audit,
- une exécution **rapide** adaptée à une vente flash.

6 Implémentation des cas d'usage

Cette section décrit les cas d'usage principaux du système de vente flash, ainsi que la logique technique mise en place pour garantir **cohérence**, **anti-survente**, **expiration automatique** et **idempotence**. L'implémentation suit un principe : **toutes les opérations critiques qui modifient le stock sont atomiques** afin d'éviter les conditions de course.

6.1 Initialisation du stock

Objectif : initialiser proprement le produit dans Redis, en créant :

- `stock:product:<id>` : la capacité totale (stock initial),
- `stock:reserved:<id>` : compteur de stock réservé, initialisé à 0.

Pourquoi c'est important ? Sans initialisation contrôlée, on risque des états incohérents (stock non défini, compteurs manquants) et donc des erreurs pendant une vente flash. L'opération est volontairement simple et **idempotente** : relancer l'initialisation doit remettre le produit dans un état sain.

Commande CLI (exemple)

```
python app.py init-stock --product 101 --total 50
python app.py status --product 101
```

Critères d'acceptation

- `total` vaut la valeur demandée (ex. 50).
- `reserved=0` et `available=total`.
- Aucun log d'incident (`metrics:incidents=0`).

6.2 Réserve atomique (Lua)

Objectif : empêcher la survente sous forte concurrence (plusieurs clients en même temps).

Le problème classique est une **race condition** :

Plusieurs threads lisent `available` au même instant, chacun pense que le stock est suffisant, puis tous réservent $\Rightarrow$ `reserved` peut dépasser `total`.

Solution : script Lua exécuté dans Redis Redis exécute un script Lua de manière **atomique**. Cela signifie qu'aucune autre commande ne peut s'intercaler pendant l'exécution du script : **check + update est inséparable**.

Le script Lua effectue en une seule opération :

- lecture de `stock:product:<id>` et `stock:reserved:<id>`,
- vérification du stock disponible :

$$available = total - reserved$$

- refus immédiat si `available < qty`,
- création de `reservation:<id>:<user>` avec **TTL**,
- incrément de `stock:reserved:<id>` (réservation effective),
- indexation dans `active:reservations` et `active:reservation:qty`,
- écriture d'un événement dans `audit:stock`.

Commande CLI (exemple)

```
python app.py reserve --product 101 --user user42
--qty 2 --ttl 10
python app.py status --product 101
```

Résultats attendus

- si succès : `reserved` augmente, `available` diminue, et une clé `reservation:*` existe avec TTL.
- si échec : aucune modification des compteurs, retour `FAIL_NO_STOCK`.

Critères d'acceptation

- **reserved** ne dépasse **jamais total**, même avec 200/2000 clients simultanés.
- Toute réservation est traçable dans `audit:stock`.

6.3 Annulation (cancel)

Objectif : libérer immédiatement le stock si le client annule avant expiration.

Logique métier L'annulation correspond à un panier abandonné volontairement. Le système doit :

- vérifier que la réservation existe,
- récupérer `qty` réservée,
- décrémenter `stock:reserved:<id>` de `qty`,
- supprimer `reservation:<id>:<user>`,
- nettoyer les index : `active:reservations` et `active:reservation:qty`,
- tracer l'action dans `audit:stock`.

Commande CLI (exemple)

```
python app.py cancel --product 101 --user user42
python app.py status --product 101
```

Critères d'acceptation

- après `cancel` : **reserved** diminue et **available** augmente.
- la réservation est supprimée et n'apparaît plus dans les index.

6.4 Expiration et libération automatique (sweeper)

Objectif : libérer automatiquement le stock si le client abandonne sans annuler (cas réel fréquent).

Problème Le TTL supprime la clé `reservation:*`, mais Redis ne décrémente pas automatiquement `stock:reserved`. Sans mécanisme complémentaire, on pourrait conserver un stock réservé fictif et perdre des ventes.

Solution : Sweeper (nettoyage périodique) Un **sweeper** scanne périodiquement les réservations actives et détecte celles expirées (clé absente). Il s'appuie sur :

- `active:reservations` (Set) : liste des identifiants `id:user`,
- `active:reservation:qty` (Hash) : permet de connaître `qty` même si `reservation:*` a expiré,
- décrémentation de `stock:reserved:<id>` et mise à jour des métriques : `metrics:abandon++`,
- ajout d'un log `EXPIRE_RELEASE` dans `audit:stock`.

Commande CLI (exemple)

```
python app.py sweep-once
python app.py metrics
python app.py audit --n 10
```

Critères d'acceptation

- après expiration + sweep : **reserved** revient à un état correct.
- **metrics:abandon** augmente et l'audit prouve la libération.

6.5 Paiement confirmé (idempotence)

Objectif : empêcher le double paiement (ex. double clic, retry réseau, re-submit).

Problème Dans le monde réel, un client peut déclencher plusieurs confirmations pour la même commande :

- rechargement de page,
- latence réseau et retry,
- double clic sur *Payer*.

Sans protection, la même commande pourrait décrémenter plusieurs fois le stock réservé ou fausser les KPI.

Solution : idempotence via clé `payment:processed:<order>` Lors de confirm, le système exécute :

```
SET payment:processed:<order> "1" NX EX <ttl>
```

- si la clé existe déjà : **paiement déjà traité** ⇒ retour `ALREADY_CONFIRMED` (aucun effet).
- si la clé est créée : **premier traitement** ⇒ finalisation :
 - décrémentation de `stock:reserved:<id>` de `qty`,
 - suppression de `reservation:<id>:<user>` si encore présente,
 - nettoyage des index actifs,
 - `metrics:sold++`,
 - audit `CONFIRM`.

Commande CLI (exemple)

```
python app.py confirm --product 101 --user user42
--order order-9001
python app.py confirm --product 101 --user user42
--order order-9001 # retry (idempotent)
python app.py metrics
```

Critères d'acceptation

- un même `order` n'augmente `metrics:sold` qu'une seule fois.
- aucune incohérence stock (réservé jamais négatif, total inchangé).
- événement `CONFIRM` présent dans `audit:stock`.

7 Gestion des cas limites

Dans un système de vente flash réel, les anomalies ne sont pas l'exception mais la norme : forte concurrence, erreurs réseau, comportements imprévisibles des utilisateurs. Cette section présente les principaux **cas limites (edge cases)** identifiés ainsi que les mécanismes techniques mis en place pour garantir la **cohérence du stock**, la **robustesse du système** et la **traçabilité des incidents**.

7.1 Survente (Race Condition)

Risque : Lorsque plusieurs clients tentent de réserver simultanément, ils peuvent lire le même stock disponible avant qu'il ne soit mis à jour, ce qui peut conduire à :

$$stock_reserved > stock_total \Rightarrow survente$$

Solution technique : atomicité via Lua La réservation est effectuée à l'aide d'un **script Lua exécuté côté Redis**. Ce script réalise en une seule opération :

- lecture du stock total et réservé,
- vérification du stock disponible,
- mise à jour du stock réservé,
- création de la réservation avec TTL.

Redis garantit que le script est exécuté **sans interruption**, ce qui élimine les conditions de course.

Preuve expérimentale : Le script `scenario_concurrency.py` montre :

- scénario SAFE (Lua) : **PASS - no oversell**,
- scénario UNSAFE (sans Lua) : **FAIL - reserved > total**.

7.2 Double paiement

Risque : Un client peut déclencher plusieurs confirmations de paiement (double clic, retry réseau, rafraîchissement page). Sans protection, une même commande pourrait être traitée plusieurs fois.

Solution : idempotence par clé de verrouillage

```
SET payment:processed:<order> 1 NX EX <ttl>
```

- si la clé existe déjà $\Rightarrow$ paiement déjà traité (aucun effet),
- sinon $\Rightarrow$ traitement unique : décrémentation du stock réservé + `metrics:sold++`.

Impact : Chaque commande est comptabilisée une seule fois, même en cas de répétition.

7.3 Requêtes répétées / retries

Risque : En environnement distribué, les requêtes peuvent être rejouées :

- timeout réseau,
- retry automatique du client,
- redémarrage du service.

Mécanismes de protection :

- une réservation déjà existante est détectée (`reservation:<id>:<user>`),
- un paiement déjà traité est bloqué par `payment:processed:<order>`,
- les opérations deviennent **idempotentes**.

Résultat : le système peut recevoir la même requête plusieurs fois sans altérer l'état global.

7.4 Abandon tardif du client

Risque : Un client ferme le navigateur ou perd la connexion sans annuler explicitement. Le stock resterait réservé indéfiniment sans mécanisme automatique.

Solution combinée : TTL + Sweeper

- chaque réservation possède un TTL (expiration automatique),
- le **sweeper** détecte les clés expirées,
- il libère le stock correspondant,
- incrémente `metrics:abandon`,
- écrit un log `EXPIRE_RELEASE` dans `audit:stock`.

Impact : le stock revient toujours à un état cohérent, même si le client disparaît.

7.5 Données manquantes ou incohérentes

Risque :

- suppression inattendue de clés,
- arrêt brutal du service,
- erreur logique ou bug.

Mécanismes de détection :

- compteur `metrics:incidents` pour signaler toute incohérence détectée,
- journal d'audit `audit:stock` retraçant chaque action (INIT, RESERVE, CONFIRM, CANCEL, EXPIRE_RELEASE),
- possibilité d'analyse post-mortem.

Objectif : assurer une **observabilité complète** du système.

7.6 Résumé

Ces mécanismes combinés garantissent que :

- le stock ne devient jamais négatif,
- aucune commande n'est comptée deux fois,
- les abandons sont automatiquement gérés,
- les erreurs sont détectées et tracées.

Le système reste donc cohérent, fiable et vérifiable même sous forte charge concurrente.

8 Installation et exécution (Windows + Docker)

Cette section décrit l'environnement nécessaire pour exécuter le projet ainsi que les étapes complètes permettant de démarrer l'application et de vérifier son bon fonctionnement.

8.1 Prérequis

Avant d'exécuter le projet, les éléments suivants doivent être installés :

- **Docker Desktop** (Windows) pour exécuter Redis dans un conteneur isolé.
- **Python 3.11 ou supérieur**.
- **pip** pour l'installation des dépendances.
- Fichiers du projet clonés localement (FlashSale/).

Installation des dépendances Python :

```
pip install -r requirements.txt
```

8.2 Démarrage du serveur Redis (Docker)

L'instance Redis utilisée pour le projet est exécutée dans un conteneur Docker :

```
docker run -d --name redis-flash -p 6379:6379 redis  
:7
```

Vérifier que le conteneur fonctionne :

```
docker ps
```

Tester Redis directement :

```
docker exec -it redis-flash redis-cli ping  
# R ponse attendue : PONG
```

8.3 Configuration de l'authentification Redis (ACL)

Pour sécuriser l'accès, un utilisateur Redis dédié est configuré :

```
docker exec -it redis-flash redis-cli  
ACL SETUSER khalid on >5002 allcommands allkeys  
ACL SETUSER default off
```

Test de connexion :

```
docker exec -it redis-flash redis-cli --user khalid  
--pass 5002 PING  
# R ponse attendue : PONG
```


8.4 Variables d'environnement (PowerShell)

Les identifiants Redis sont fournis à l'application via des variables d'environnement :

```
$env:REDIS_USERNAME="khalid"  
$env:REDIS_PASSWORD="5002"
```

Ces variables sont lues automatiquement par config.py.

8.5 Vérification de l'application (Health Check)

Se placer dans le dossier du service :

```
cd 1_Service
```

Tester la connectivité :

```
python app.py ping
```

Sortie attendue :

```
Settings(redis=127.0.0.1:6379/db0, auth=acl-auth (  
    username+password), ...)  
PONG
```

Cette commande confirme :

- la connexion à Redis,
- la validité des identifiants ACL,
- le bon fonctionnement de la configuration.

8.6 Commandes principales de démonstration

Initialiser le stock :

```
python app.py init-stock --product 101 --total 50
```

Réserver un produit :

```
python app.py reserve --product 101 --user user42  
--qty 2 --ttl 10
```

Voir l'état du stock :

```
python app.py status --product 101
```

Afficher les métriques :

```
python app.py metrics
```

Afficher l'audit :

```
python app.py audit --n 10
```

9 Tests, simulation de charge et preuves

Afin de démontrer la robustesse du système face à la concurrence et de prouver l'absence de survente, des scénarios de tests automatisés ont été mis en place. Ces tests simulent des clients parallèles et mesurent les performances, la cohérence du stock et les indicateurs métiers.

9.1 Simulation multi-clients (`simulate_clients.py`)

Objectif : reproduire le comportement réel d'une vente flash où de nombreux clients interagissent simultanément avec le système.

Chaque client simulé peut :

- réserver un produit,
- confirmer le paiement,
- annuler la réservation,
- abandonner le panier (expiration TTL).

Le script génère des **KPI (Key Performance Indicators)** :

- nombre de confirmations,
- taux d'abandon,
- échecs par manque de stock,
- latence des opérations,
- cohérence finale du stock.

Commande de test :

```
python 2_Simulation/simulate_clients.py --product
101 --stock 50 --clients 200 --ttl 8 \
--confirm-rate 0.6 --cancel-rate 0.1 --abandon-rate
0.3 --sweeper-interval 2
```

Signification des paramètres :

- `-clients` : nombre total de clients simulés.
- `-confirm-rate` : probabilité qu'un client confirme son achat.
- `-cancel-rate` : probabilité d'annulation volontaire.
- `-abandon-rate` : probabilité d'abandon (TTL).
- `-ttl` : durée de validité d'une réservation.
- `-sweeper-interval` : fréquence de libération automatique.

9.2 Comparatif SAFE vs UNSAFE (`scenario_concurrency.py`)

Objectif : démontrer expérimentalement l'importance de l'atomicité.

Deux scénarios sont exécutés :

- **SAFE** : réservation via script Lua atomique.
- **UNSAFE** : approche naïve (vérification puis mise à jour séparées).

Commande :

```
python 2_Simulation/scenario_concurrency.py --
    product 101 --stock 20 --clients 200 --ttl 8 --
    unsafe-delay-ms 5
```

Résultat attendu :

- SAFE : stock cohérent, aucune survente.
- UNSAFE : dépassement du stock total, démontrant une condition de course.

9.3 Interprétation des résultats

- **SAFE (Lua)** : `reserved` ne dépasse jamais `total` $\Rightarrow$ **PASS**.
- **UNSAFE (naïf)** : `reserved > total` ou `available < 0` $\Rightarrow$ **FAIL (survente)**.

Ces tests constituent une **preuve expérimentale** que l'architecture choisie empêche la survente même sous forte charge concurrente.

9.4 Logs et preuves

Toutes les exécutions importantes sont enregistrées dans :

`2_Simulation/results_logs.txt`

Ce fichier contient :

- les commandes exécutées,
- les résultats numériques (KPI),
- les vérifications PASS/FAIL,
- la preuve de cohérence finale du stock.

Il constitue un **artefact de preuve** démontrant la validité scientifique et technique de la solution.

10 Planification du projet (Diagramme de Gantt)

Objectif : présenter la planification temporelle du projet et la répartition des tâches sur les sprints afin de démontrer une gestion structurée conforme aux pratiques Agile.

Le diagramme de Gantt ci-dessous illustre :

- la progression incrémentale du projet,
- la logique d'enchaînement des tâches,
- la répartition équilibrée entre conception, implémentation, tests et validation.

Chaque sprint correspond à une phase clé :

- **Sprint 1 (W1)** : Analyse métier, conception du modèle Redis, préparation de l'environnement (Docker + ACL).
- **Sprint 2 (W2)** : Implémentation de la réservation atomique (TTL + Lua), annulation et libération.
- **Sprint 3 (W3)** : Idempotence du paiement, métriques, audit, scripts de simulation.

- **Sprint 4 (W4)** : Tests de concurrence SAFE vs UNSAFE, preuves expérimentales, finalisation du rapport.

SPRINT	WEEK 1	WEEK 2	WEEK 3	WEEK 4
SPRINT 1	S1 : Analyse & modèle Redis			
SPRINT 2		S2 : TTL + Lua + Cancel		
SPRINT 3			S3 : Paiement + KPI	
SPRINT 4				S4 : Simulation + Rapport

FIGURE 9 — Planning du projet sous forme de diagramme de Gantt — organisation par sprints

Ce planning met en évidence une **gestion de projet structurée**, avec livraison progressive de fonctionnalités démontrables à la fin de chaque sprint.

11 Conclusion

Ce projet a permis de concevoir et de valider une **architecture robuste de gestion de stock pour une vente flash**, capable de fonctionner sous forte concurrence sans provoquer de survente. La solution proposée démontre qu'un système distribué peut rester cohérent en combinant intelligemment les mécanismes offerts par Redis.

Les principaux apports techniques sont :

- **Redis (NoSQL clé-valeur)** : haute performance, faible latence et simplicité d'architecture adaptée aux charges élevées ;
- **Scripts Lua atomiques** : suppression des conditions de course grâce à une opération *check+update* exécutée côté serveur ;
- **TTL (Time-To-Live)** : gestion automatique de l'expiration des paniers et libération du stock abandonné ;
- **Idempotence du paiement** : prévention du double traitement d'une transaction via des verrous logiques ;
- **Métriques et audit** : traçabilité complète des opérations, mesure du taux d'abandon et preuve expérimentale de cohérence.

Les tests de simulation concurrente constituent une **preuve expérimentale forte** :

- Le scénario **SAFE** (réservation atomique Lua) maintient toujours

$$reserved \leq total \quad et \quad available \geq 0$$

ce qui garantit l'absence de survente.

- Le scénario **UNSAFE** (approche naïve) montre des cas où `reserved > total` et `available < 0`, illustrant les risques réels d'une mauvaise gestion de la concurrence.

Ainsi, ce projet ne se limite pas à une implémentation fonctionnelle : il démontre une **maîtrise des problématiques de cohérence en environnement distribué**, en s'appuyant sur des preuves mesurables, des métriques et une méthodologie d'ingénierie logicielle structurée.

Conclusion finale : la combinaison *Redis + atomicité Lua + TTL + idempotence* constitue une solution efficace, scalable et professionnelle pour sécuriser un système de vente flash contre la survente.